## Solution using open source tools

## Q1: KPI that the business should track

**Business KPIs:**

| KPI | Description |
|---|---|
| **Model Accuracy (mAP)** | Mean Average Precision (especially for object detection of scratches & dents). |
| **Precision/Recall per class** | Helps understand whether one type of damage is under- or over-reported. |
| **Damage Detection Rate** | % of images in which damage is detected (compared to actual damaged ones). |
| **False Positives Rate** | Damages detected when there were none (important to not reduce resale value unfairly). |
| **Time to Inference** | How long it takes to get prediction results in production. |
| **Drift Score** | Change in input image quality or domain (lighting, camera, angles). |
| **Deployment Frequency** | Frequency of safe automated deployments with new data annotations. |

---

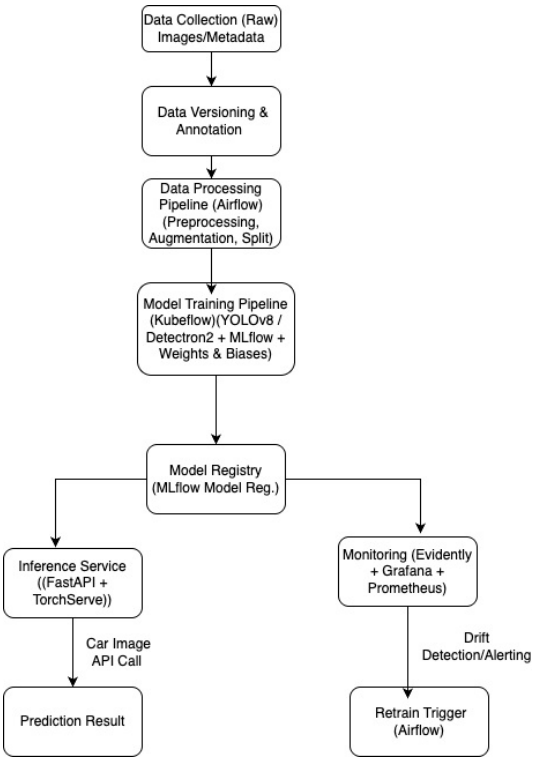
## Q2: Why MLOps over just a simple model?

| Benefit | Description |
|---|---|
| **Scalability** | Handles growth in dataset, experiments, models automatically. |
| **Reproducibility** | Tracks each experiment and data used. |
| **Automation** | Automates data ingestion, retraining, deployment, monitoring. |
| **Continuous Improvement** | Auto-training on new data and drift handling helps improve over time. |
| **Monitoring & Alerting** | Detects performance degradation, data drift, and allows proactive action. |
| **CI/CD Integration** | Integrates seamlessly with version control and production environments. |

---

## Q3: ML System Design Diagram

Here’s a high-level architecture diagram of the **MLOps system** for damage detection.



System Design

---

## Q4: Reason for Choosing the Tools

| Tool | Reason |
|---|---|
| **Label Studio** | Easy-to-use UI for annotating images (scratch, dent, none). |
| **DVC** | Data versioning and reproducibility. |
| **YOLOv8 / Detectron2** | State-of-the-art object detection models for real-time detection. |
| **MLflow** | Experiment tracking, model registry, and lifecycle management. |
| **Weights & Biases** | Visualize training metrics, hyperparameter tuning. |
| **Airflow** | Orchestrate data + training + retraining pipelines. |
| **Kubeflow Pipelines** | Scalable ML pipeline for training models on Kubernetes. |
| **FastAPI + TorchServe** | Lightweight, fast model serving. |
| **Evidently AI** | Model performance + data drift detection. |
| **Prometheus + Grafana** | Real-time metrics, monitoring, and alerting. |

---

## Q5: Workflow End to End

---

### Step 1: Data and Model Experimentation

- **Tools**: Label Studio, DVC, YOLOv8/Detectron2, MLflow, W&B
- **Steps**:

1. Annotate images: scratch, dent, or none (use Label Studio).
2. Version datasets using DVC.
3. Train object detection model (YOLOv8).
4. Track experiments (MLflow), log metrics (W&B).
5. Save the best model in the MLflow Registry.

---

### Step 2: Automation of Data Pipeline

- **Tools**: Apache Airflow + DVC + Python scripts
- **Steps**:
  1. Fetch newly uploaded images.
  2. Apply data preprocessing and augmentation.
  3. Store versioned data in cloud bucket (S3/GCS) and metadata in DVC.
  4. Trigger training pipeline.

---

### Step 3: Automation of Training Pipeline

- **Tools**: Kubeflow Pipelines + MLflow + Docker
- **Steps**:
  1. Triggered by new data or drift signal.
  2. Fetch data and config from DVC + Git.
  3. Run model training in containers.
  4. Log experiments and register model to MLflow.

---

### Step 4: Automation of Inference Pipeline

- **Tools**: FastAPI + TorchServe + Docker + Kubernetes
- **Steps**:
  1. Deployed as REST endpoint.
  2. Users upload images → model detects type of damage.
  3. Return prediction as scratch/dent/none.
  4. Log input/output for monitoring and retraining.

---

### Step 5: Continuous Monitoring Pipeline

- **Tools**: Evidently AI + Prometheus + Grafana
- **Steps**:
  1. Monitor incoming image quality (e.g., lighting, size).
  2. Compare current input with training data stats.
  3. Detect drift in input data or drop in accuracy.
  4. Visualize metrics via Grafana.

---

### Model Drift Detected (e.g., poor lighting):

- **Triggered Component**: Evidently + Airflow
- **If Drift Score > Threshold**:
  - Airflow triggers retraining.
  - Use augmented lighting samples or apply lighting correction in preprocessing.
  - Re-train model and deploy.

---

### New Annotated Data Available:

- **Triggered Component**: Airflow + Kubeflow
- **Action**:
  - Labelled data added → committed with DVC → triggers retraining DAG.
  - **Runs full training, logs metrics, compares performance, and if better, promotes model to production.**

Here's a **template repository structure** for the end-to-end MLOps system to detect car damages (scratches and dents) from images using object detection, covering everything from experimentation to deployment and monitoring.

---

**used-car-damage-detector-mlops/**

```
used-car-damage-detector-mlops/
│
├──  data/
│   ├── raw/                    # Raw images (from source)
│   ├── processed/              # Preprocessed images and annotations
│   └── annotations/            # COCO/Yolo format labels
```

```
│
├── notebooks/
│   ├── EDA.ipynb                # Exploratory Data Analysis
│   └── Experiment_Tracking.ipynb # For initial experiments
│
├── src/
│   ├── data/
│   │   ├── preprocess.py        # Image preprocessing logic
│   │   └── augment.py           # Data augmentation techniques
│   │
│   ├── model/
│   │   ├── model.py             # Model architecture (e.g., YOLOv5/YOLOv8)
│   │   └── train.py             # Training loop, logging
│   │
│   ├── utils/
│   │   ├── metrics.py           # Custom metrics (e.g., mAP, IoU)
│   │   └── helpers.py           # Helper functions
│   │
│   └── inference/
│       ├── predictor.py         # For running inference on images
│       └── visualize.py         # Annotated output images
│
├── pipelines/
│   ├── data_pipeline.py         # Prefect/KubeFlow pipeline for data
│   ├── training_pipeline.py     # Orchestration of model training
│   ├── inference_pipeline.py    # Batch/real-time inference
│   └── monitoring_pipeline.py   # Drift detection, alerting
│
├── configs/
│   ├── config.yaml              # Model, data, training configs
│   └── drift_config.yaml        # Thresholds and settings for drift
│
├── tests/
│   ├── test_model.py            # Unit tests for model components
│   └── test_data.py             # Unit tests for preprocessing
│
├── deployment/
│   ├── docker/
│   │   ├── Dockerfile           # Docker setup for the service
│   ├── k8s/
│   │   ├── deployment.yaml      # Kubernetes deployment
│   │   └── service.yaml         # Kubernetes service config
│   ├── app.py                   # FastAPI or Flask app for inference
│   └── requirements.txt         # Python dependencies
│
├── monitoring/
│   ├── drift_detector.py        # Use Evidently or custom solution
│   ├── prometheus_config.yml    # Metrics collection
│   └── alerting.py              # Alert trigger to retrain model
│
├── ci_cd/
│   ├── github_actions.yml       # GitHub Actions or GitLab CI/CD
│   └── dvc.yaml                 # DVC pipeline for reproducibility
│
├── mlflow/                    # MLflow config and run tracking
│   └── mlruns/                  # Local run metadata
│
├── .gitignore
├── README.md
└── setup.py
```

---

## Solution using managed service (Amazon Sagemaker)

Below is a comprehensive solution to address the problem statement using **Amazon SageMaker** to build an end-to-end MLOps system for detecting scratches, dents, or no damage in used car images. The solution covers system design, KPIs, MLOps advantages, architecture, tool choices, and a detailed workflow, including handling drift and additional data.

---

### Q1. System Design: KPIs the Business Should Track

The business should track the following **Key Performance Indicators (KPIs)** to evaluate the effectiveness of the object-detection model and its impact on pricing strategy:

1. **Model Performance Metrics**:
   - **Precision, Recall, F1-Score** for each class (scratches, dents, none) to ensure accurate detection.
   - **Mean Average Precision (mAP)** at IoU=0.5 to measure object-detection accuracy across classes.
   - **False Negative Rate (FNR)** to minimize missed damages, as undetected damages could lead to incorrect pricing.
2. **Business Impact Metrics**:
   - **Pricing Accuracy**: Percentage of cars priced within an acceptable range of manual inspection estimates.
   - **Revenue Impact**: Increase in resale value accuracy leading to higher margins or customer trust.

    - **Operational Efficiency**: Reduction in manual inspection time (e.g., hours saved per car).
3. **System Reliability Metrics**:
    - **Model Latency**: Time taken to process an image (aim for <1 second for real-time use).
    - **Uptime**: Percentage of time the inference pipeline is available (target >99.9%).
    - **Drift Detection Rate**: Frequency and severity of data drift incidents resolved.
4. **Data Quality Metrics**:
    - **Annotation Coverage**: Percentage of images with high-quality labels.
    - **Label Consistency**: Agreement rate among annotators for labeled damages.

These KPIs align with the goal of automating damage detection to scale operations while maintaining pricing accuracy.

---

## Q2. System Design: Advantages of Building an MLOps System

Building an **MLOps system** instead of a simple model provides the following advantages:

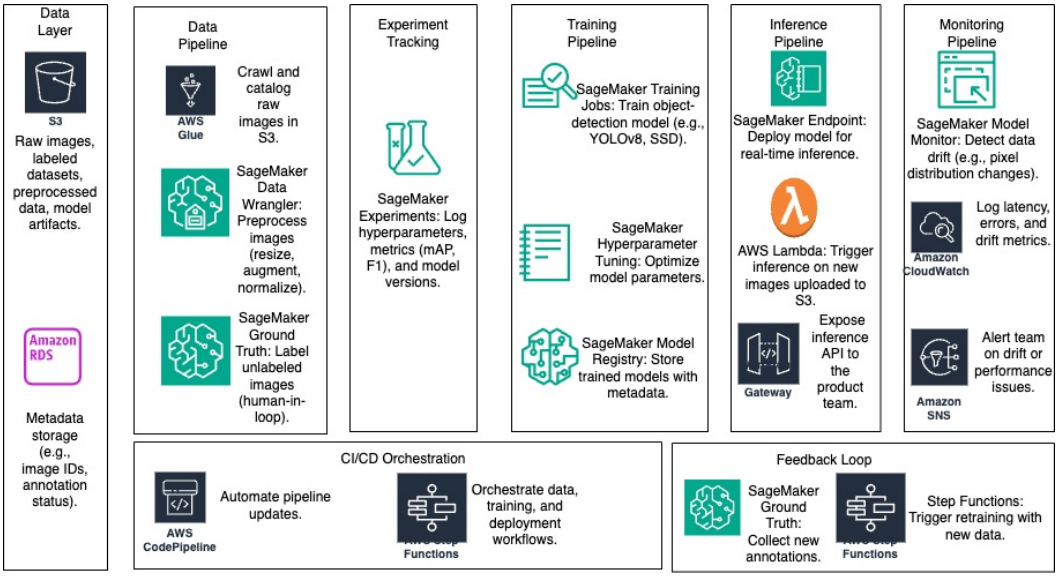
1. **Scalability**: Automates data preprocessing, training, and deployment, allowing the system to handle millions of images as the company grows.
2. **Reproducibility**: Tracks experiments (e.g., hyperparameters, datasets) to ensure consistent results and avoid manual errors.
3. **Continuous Improvement**: Enables retraining with new annotated data to improve model accuracy over time.
4. **Monitoring and Maintenance**: Detects data drift (e.g., poor lighting) and model degradation, triggering alerts or retraining to maintain performance.
5. **Automation**: Reduces human intervention in pipelines, lowering operational costs and speeding up deployment.
6. **Compliance and Governance**: Logs model versions, data lineage, and performance metrics for auditability, critical for business trust.
7. **Fault Tolerance**: Handles failures (e.g., pipeline crashes) gracefully with retries and monitoring, ensuring reliability.

An MLOps system ensures the model remains robust, adaptable, and aligned with business needs in a production environment.

---

## Q3. System Design: ML System Architecture

Below is the architecture for a complete MLOps system using Amazon SageMaker, covering experiment tracking, automated pipelines, deployment, and monitoring.




Sagemaker System Design

```
Data Layer:
- Amazon S3: Raw images, labeled datasets, preprocessed data, model artifacts.
- Amazon RDS: Metadata storage (e.g., image IDs, annotation status).

Data Pipeline:
- AWS Glue: Crawl and catalog raw images in S3.
- SageMaker Data Wrangler: Preprocess images (resize, augment, normalize).
- SageMaker Ground Truth: Label unlabeled images (human-in-loop).

Experiment Tracking:
- SageMaker Experiments: Log hyperparameters, metrics (mAP, F1), and model versions.
```

```
Training Pipeline:
- SageMaker Training Jobs: Train object-detection model (e.g., YOLOv8, SSD).
- SageMaker Hyperparameter Tuning: Optimize model parameters.
- SageMaker Model Registry: Store trained models with metadata.

Inference Pipeline:
- SageMaker Endpoint: Deploy model for real-time inference.
- AWS Lambda: Trigger inference on new images uploaded to S3.
- Amazon API Gateway: Expose inference API to the product team.

Monitoring Pipeline:
- SageMaker Model Monitor: Detect data drift (e.g., pixel distribution changes).
- Amazon CloudWatch: Log latency, errors, and drift metrics.
- SNS: Alert team on drift or performance issues.

CI/CD and Orchestration:
- AWS CodePipeline: Automate pipeline updates.
- AWS Step Functions: Orchestrate data, training, and deployment workflows.

Feedback Loop:
- SageMaker Ground Truth: Collect new annotations.
- Step Functions: Trigger retraining with new data.
```

**Note**: Since I cannot render diagrams directly, the above is a textual description of the architecture. Tools like Lucidchart or Draw.io can be used to visualize this flow, with S3 as the central data hub, SageMaker components for ML tasks, and AWS services for automation and monitoring.

---

### Q4. System Design: Reasons for Choosing Specific Tools

The tools were chosen for their integration, scalability, and alignment with the use case:

1. **Amazon S3**:
   - **Reason**: Scalable, cost-effective storage for millions of images and model artifacts. Supports versioning and lifecycle policies to manage data efficiently.
   - **Use Case**: Stores raw images, preprocessed datasets, and trained models.
2. **Amazon SageMaker**:
   - **Reason**: Fully managed ML platform with built-in tools for training, tuning, deployment, and monitoring. Supports popular frameworks (e.g., PyTorch for YOLOv8).
   - **Use Case**: Handles experiment tracking, model training, hyperparameter optimization, and real-time inference.
3. **SageMaker Ground Truth**:
   - **Reason**: Streamlines annotation of unlabeled images with human-in-loop workflows, improving label quality and enabling continuous data updates.
   - **Use Case**: Labels new or unlabeled images for training and retraining.
4. **AWS Glue**:
   - **Reason**: Automates data cataloging and ETL (Extract, Transform, Load) for large-scale image datasets in S3.
   - **Use Case**: Organizes raw images and metadata for preprocessing.
5. **AWS Step Functions**:
   - **Reason**: Orchestrates complex workflows (data prep, training, deployment) with error handling and retries, ensuring pipeline reliability.
   - **Use Case**: Coordinates end-to-end MLOps workflows.
6. **AWS Lambda**:
   - **Reason**: Serverless compute for event-driven tasks, reducing costs for sporadic inference triggers.
   - **Use Case**: Triggers preprocessing or inference when new images are uploaded.
7. **Amazon API Gateway**:
   - **Reason**: Provides a secure, scalable API to expose the model to the product team, with throttling and authentication.
   - **Use Case**: Delivers damage detection results to pricing applications.
8. **SageMaker Model Monitor and CloudWatch**:
   - **Reason**: Detects data drift (e.g., lighting issues) and logs system metrics (latency, errors) for proactive maintenance.
   - **Use Case**: Ensures model reliability in production.
9. **AWS CodePipeline**:
   - **Reason**: Automates CI/CD for pipeline updates, ensuring rapid iteration and deployment.
   - **Use Case**: Updates training or inference code with new features.
10. **Amazon SNS**:
    - **Reason**: Simple notification service for alerting teams about drift or pipeline failures.
    - **Use Case**: Notifies stakeholders for manual intervention.

These tools are tightly integrated within AWS, reducing latency and complexity while supporting scalability for millions of images.

---

### Q5. Workflow of the Solution

Below is the end-to-end workflow for building the MLOps system, including tools, their connections, and actions for drift and new data scenarios.

**1. Data and Model Experimentation**

- **Steps**:
  1. **Data Ingestion**:
     - Store raw images in **Amazon S3** (e.g., `s3://used-cars/raw/`).
     - Use **AWS Glue** to crawl S3 and create a data catalog with metadata (e.g., image ID, upload date).
  2. **Labeling**:
     - Use **SageMaker Ground Truth** to annotate unlabeled images for scratches, dents, or none.
     - Store annotations in S3 (e.g., `s3://used-cars/labels/`).
  3. **Preprocessing**:
     - Use **SageMaker Data Wrangler** to resize images (e.g., 416x416 for YOLOv8), normalize pixel values, and apply augmentations (e.g., rotation, flip).
     - Save preprocessed datasets in S3 (e.g., `s3://used-cars/processed/`).
  4. **Experimentation**:
     - Train an object-detection model (e.g., YOLOv8) using **SageMaker Training Jobs** with PyTorch.
     - Log experiments (hyperparameters, mAP, F1) in **SageMaker Experiments**.
     - Tune hyperparameters (e.g., learning rate, batch size) with **SageMaker Hyperparameter Tuning**.
     - Store the best model in **SageMaker Model Registry**.
- **Tools and Connections**:
  - S3 → Glue → Ground Truth → Data Wrangler → SageMaker Training → Experiments → Model Registry.
  - Data flows from S3 to preprocessing, then to training, with metadata logged for reproducibility.

**2. Automation of Data Pipeline**

- **Steps**:
  1. **Trigger**: New images uploaded to S3 (`s3://used-cars/raw/`).
  2. **Cataloging**: **AWS Lambda** triggers **AWS Glue** to update the data catalog.
  3. **Labeling**: If unlabeled, **Step Functions** routes images to **SageMaker Ground Truth** for annotation.
  4. **Preprocessing**: **SageMaker Data Wrangler** processes images (resize, augment) and saves them to S3 (`s3://used-cars/processed/`).
- **Tools and Connections**:
  - S3 → Lambda → Glue → Step Functions → Ground Truth → Data Wrangler → S3.
  - **Step Functions** orchestrates the flow, ensuring unlabeled images are annotated before preprocessing.

**3. Automation of Training Pipeline**

- **Steps**:
  1. **Trigger**: New preprocessed data or scheduled retraining (e.g., weekly via **CloudWatch Events**).
  2. **Training**: **Step Functions** launches a **SageMaker Training Job** with the latest dataset from S3.
  3. **Evaluation**: Compute mAP and F1 on a validation set, logged to **SageMaker Experiments**.
  4. **Model Approval**: If metrics exceed thresholds (e.g., mAP > 0.85), register the model in **SageMaker Model Registry** via **Step Functions**.
  5. **CI/CD**: **AWS CodePipeline** updates training scripts if new features are added (e.g., new augmentation).
- **Tools and Connections**:
  - S3 → CloudWatch Events → Step Functions → SageMaker Training → Experiments → Model Registry → CodePipeline.
  - Training is fully automated, with performance metrics driving model registration.

**4. Automation of Inference Pipeline**

- **Steps**:
  1. **Deployment**: Deploy the approved model from **SageMaker Model Registry** to a **SageMaker Endpoint**.
  2. **Trigger**: New image uploaded to S3 triggers **AWS Lambda**.
  3. **Inference**: Lambda invokes the SageMaker Endpoint, which returns predictions (scratches, dents, none).
  4. **API Access**: **Amazon API Gateway** exposes the endpoint to the product team for pricing integration.
  5. **Storage**: Save predictions in S3 (`s3://used-cars/predictions/`) and metadata in **Amazon RDS**.
- **Tools and Connections**:
  - Model Registry → SageMaker Endpoint → Lambda → API Gateway → S3 → RDS.
  - Inference is event-driven, with API Gateway ensuring secure access.

**5. Continuous Monitoring Pipeline**

- **Steps**:
  1. **Drift Detection**: **SageMaker Model Monitor** analyzes incoming images for data drift (e.g., pixel distribution shifts due to lighting).
  2. **Performance Monitoring**: **CloudWatch** logs endpoint latency, error rates, and prediction distributions.
  3. **Alerts**: If drift or errors exceed thresholds, **Amazon SNS** notifies the team.
  4. **Logging**: Store monitoring logs in **CloudWatch Logs** for analysis.
- **Tools and Connections**:
  - SageMaker Endpoint → Model Monitor → CloudWatch → SNS.
  - Monitoring runs continuously, with alerts enabling rapid response.

**Handling Specific Conditions**

1. **Sudden Increase in Drift Due to Poor Lighting**
   - **Component Triggered**: **SageMaker Model Monitor** detects drift by comparing input image statistics

(e.g., pixel intensity histograms) to the training baseline.
    - **Action if Drift is Detected**:
        - Model Monitor logs drift metrics to **CloudWatch**.
        - If drift is minor, **SNS** sends a low-priority alert for review.
    - **Action if Drift Exceeds Threshold** (e.g., KL divergence > 0.1):
        - **Step Functions** triggers a retraining pipeline:
            - Collect recent images with drift.
            - Route to **SageMaker Ground Truth** for re-annotation if needed.
            - Preprocess with updated lighting augmentations (e.g., brightness jitter).
            - Retrain using **SageMaker Training Job**.
            - Deploy the new model to the **SageMaker Endpoint** if metrics improve.
        - **SNS** notifies the team of the retraining outcome.
2. **Additional Annotated Data Available**
    - **Component Triggered**: **AWS Step Functions** detects new annotations in S3 (`s3://used-cars/labels/`).
    - **Action**:
        - **Data Pipeline**: **SageMaker Data Wrangler** preprocesses new annotated images and merges them with the existing dataset in S3.
        - **Training Pipeline**: **Step Functions** triggers a **SageMaker Training Job** with the updated dataset.
        - **Evaluation**: Log metrics (mAP, F1) to **SageMaker Experiments**.
        - **Deployment**: If the new model outperforms the current one (e.g., mAP improves by 0.05), register it in **SageMaker Model Registry** and deploy to the **SageMaker Endpoint**.
        - **SNS** notifies the team of the update.

---

### Summary

This solution leverages **Amazon SageMaker** and AWS services to build a scalable, automated MLOps system for detecting car damages (scratches, dents, none). The architecture ensures:

- **Experimentation**: Tracks model versions and metrics for reproducibility.
- **Automation**: Streamlines data, training, and inference pipelines with **Step Functions** and **Lambda**.
- **Monitoring**: Detects drift and performance issues with **Model Monitor** and **CloudWatch**.
- **Adaptability**: Handles new data and drift through retraining workflows.